



DATA WAREHOUSING AND BUSINESS INTELLIGENCE

Assignment 1

IT23205406
K. A. JAYATHILAKA
Y3. S2. 01.02

Table of Contents

Task 1: Data Set Selection	2
Task 2: Preparation of Data Sources	4
Task 3: Solution Architecture.....	8
Task 4: Data Warehouse Design & Development.....	10
Task 5: ETL Development (SSIS)	18
Task 6: ETL Development – Accumulating Snapshot Fact Table	27

Task 1: Data Set Selection

1.1 Dataset Description

For this assignment, I have selected the "Synthetic Logistics Operations Database (2022-2024)" sourced from Kaggle. This dataset provides a complete, highly normalized operational database (OLTP) simulating a Class 8 trucking company. It contains over 85,000 records distributed across 14 interconnected tables, capturing daily logistics operations such as driver assignments, fleet tracking, fuel purchases, and delivery performance.

Data Set Link: [logistics-operations-database](#)

1.2 Justification

This dataset perfectly aligns with the requirements set forth in the assignment specification:

- **Novel OLTP Scenario:** The dataset represents a realistic, real-world supply chain domain. It is an operational database natively tracking daily transactions, ensuring it is a true OLTP source rather than a pre-aggregated OLAP cube.
- **Sufficient Timeframe & Volume:** The data spans a complete three-year period from January 2022 through December 2024. With over 131,000 fuel purchase records, 114,000 delivery events, and 57,000 load transactions, it significantly exceeds the one-year minimum requirement, eliminating the need to auto-generate synthetic data.
- **DWBI Suitability & Richness: * Dimensions:** The database provides distinct reference entities (Drivers, Trucks, Trailers, Customers, Facilities, Routes) that will serve as rich dimensions.
 - **Facts:** The operational transactions (Trips, Fuel_Purchases, Delivery_Events) contain measurable metrics (fuel gallons, costs, distances, durations) ideal for fact tables.
 - **Hierarchies & Enrichment:** The data includes geographic components (e.g., Facilities located in 25+ major US cities) and time-series data, allowing for the creation of spatial and temporal hierarchies in the final SSAS cubes.

Below is the Entity-Relationship Diagram representing the highly normalized OLTP structure of the source dataset. This structure will be transformed into a dimensional model in subsequent tasks.

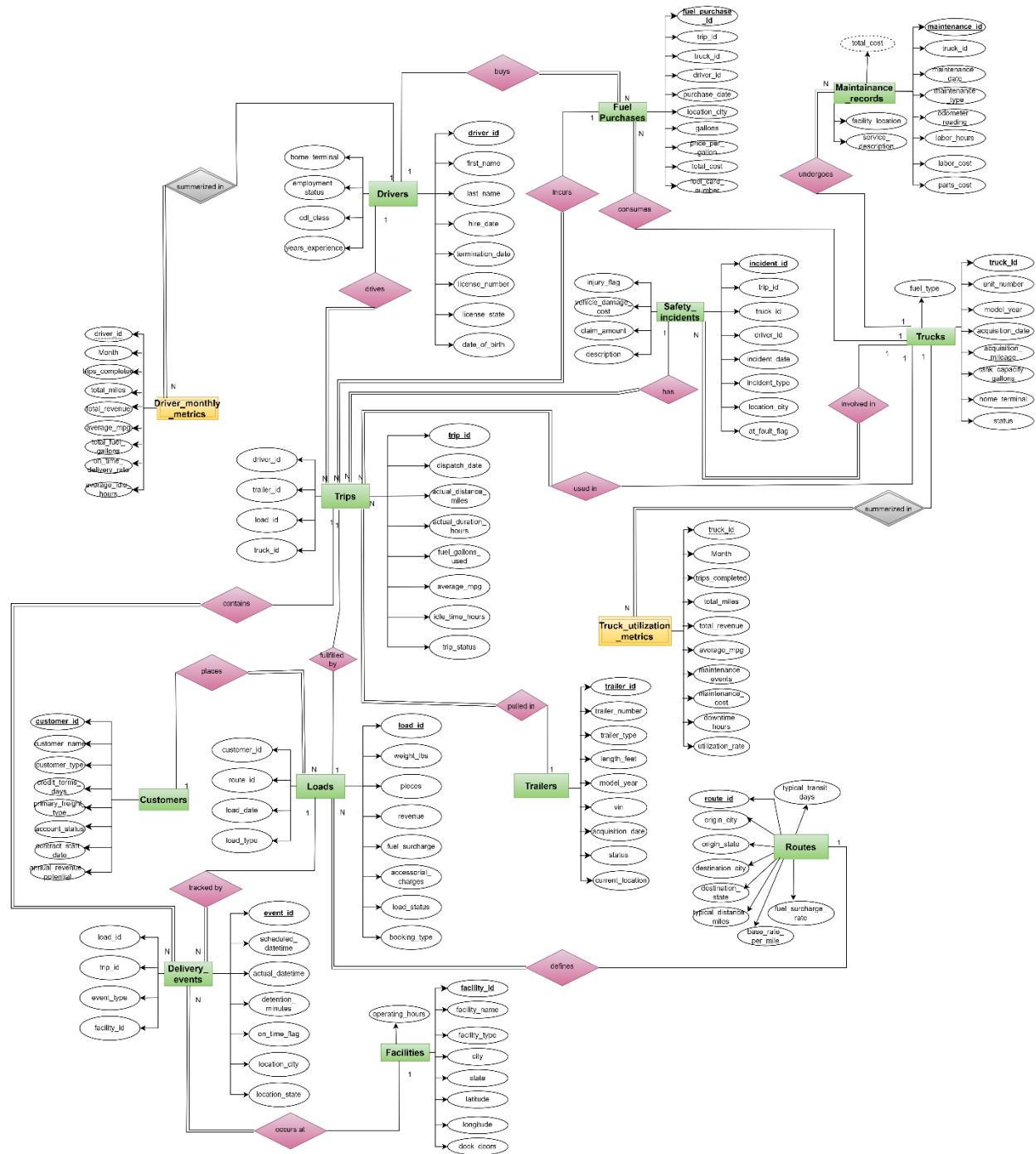


Figure 1 - ER Diagram

Task 2: Preparation of Data Sources

2.1 Data Source Strategy & Separation

To accurately simulate a real-world enterprise Business Intelligence environment and satisfy the requirement of extracting from multiple source types, the Kaggle "Logistics Operations Database" has been logically separated into two distinct source systems.

In a typical logistics company, core freight operations are managed in a central relational database, while personnel and client relationship data often reside in separate Human Resources (HR) or Customer Relationship Management (CRM) systems. Therefore, the data has been prepared into the following two formats:

Source 1: Primary OLTP Database (Microsoft SQL Server)

The bulk of the operational data has been loaded into a SQL Server relational database. This system acts as the primary source for our high-volume transactional facts and core fleet dimensions.

- Format: SQL Server Database Tables.
- Information Available:
 - Transactional Data (Future Facts): Trips, Loads, Delivery_Events, Fuel_Purchases, Maintenance_Records, and Safety_Incidents.
 - Operational Fleet Data (Future Dimensions): Trucks, Trailers, Routes, and Facilities.

Source 2: External Flat Files (CSV)

To demonstrate the extraction and integration of disparate data types, client and personnel data have been retained as flat files, simulating exports from external CRM and HR software platforms.

- Format: Comma-Separated Values (.csv) files.
- Information Available:
 - customers.csv: Contains customer account information, billing details, and contract terms.
 - drivers.csv: Contains driver demographics, licensing information, and employment history.

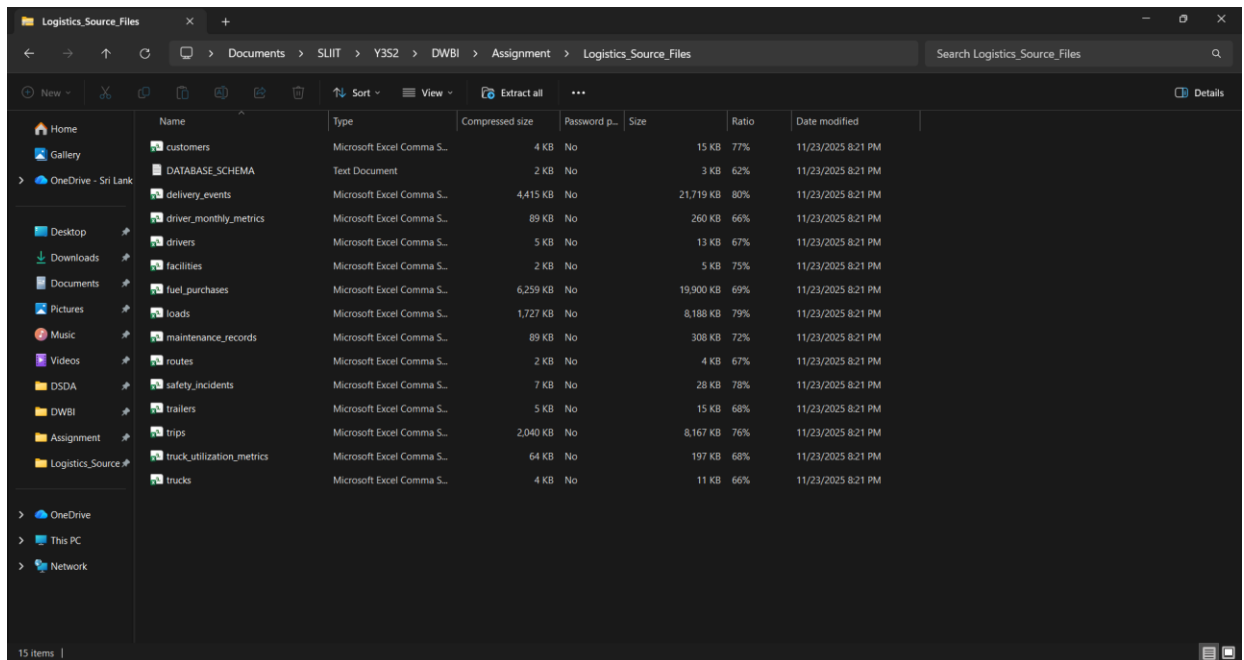


Figure 2 - Step 1 - Extracted raw source files from the Kaggle dataset.

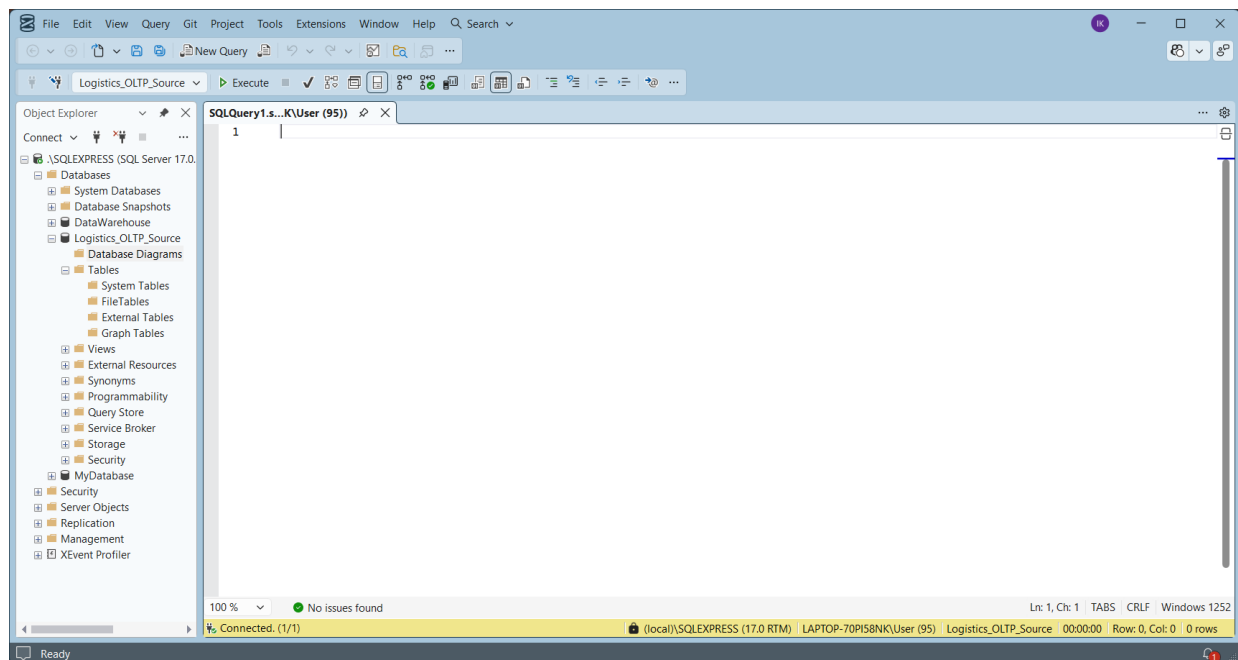


Figure 3 - Step 2 - Created the primary OLTP source database in SQL Server.

Import Flat File 'Logistics_OLTP_Source'

Modify Columns

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Modify Columns
This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
trip_id	nvarchar(50)	☒	☐	
load_id	nvarchar(50)	☐	☐	
driver_id	nvarchar(50)	☐	☒	
truck_id	nvarchar(50)	☐	☒	
trailer_id	nvarchar(50)	☐	☒	
dispatch_date	date	☐	☐	
actual_distance_miles	smallint	☐	☐	
actual_duration_hours	float	☐	☐	
fuel_gallons_used	float	☐	☐	
average_mpg	float	☐	☐	
idle_time_hours	float	☐	☐	
trip_status	nvarchar(50)	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges)
No Range

< Previous

Next >

Cancel

Figure 4 - Step 3 - Utilizing the Import Flat File wizard to load operational CSV data into SQL Server tables.

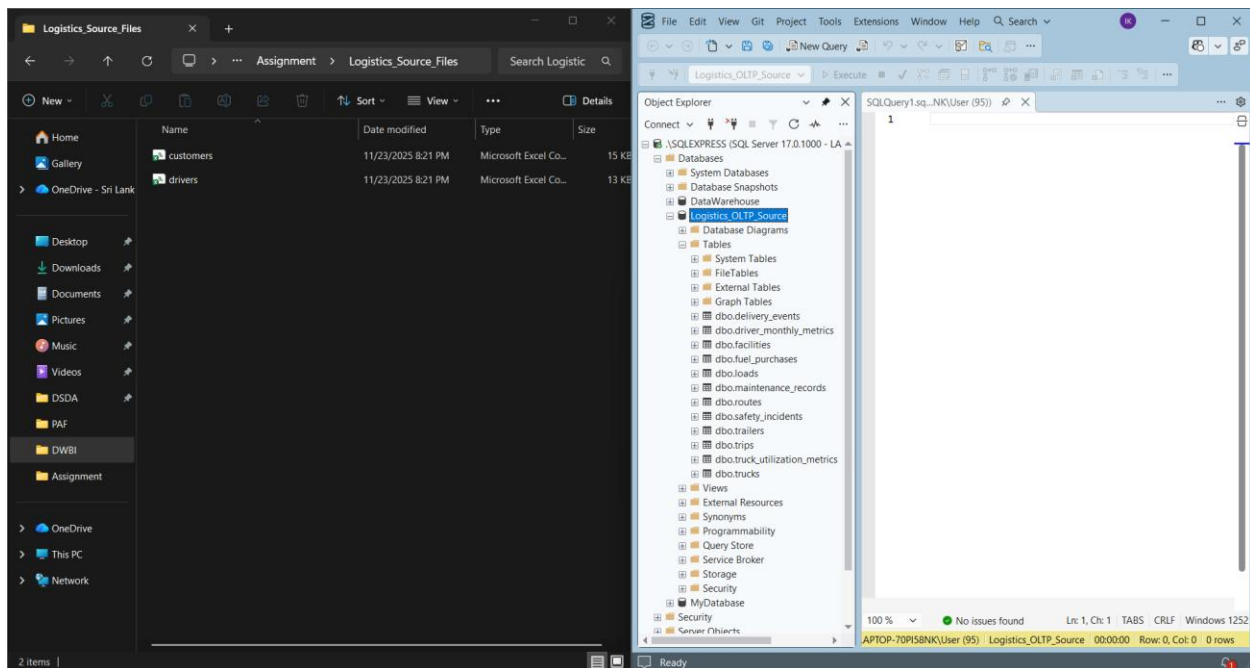


Figure 5 - Step 4 - Final Source Architecture. Operational data successfully loaded into SQL Server while Customer and Driver reference data remain as external CSV files to satisfy the multi-sources.

2.2 Data Enrichment and Hierarchy Planning

By separating the customers.csv file from the main database, we can demonstrate data enrichment during the ETL process. The customers.csv file contains descriptive attributes such as customer_type (e.g., Dedicated, Spot, Contract) and primary_industry (e.g., Retail, Food/Beverage, Automotive).

During the SSIS data extraction phase, these flat-file attributes will be integrated with the SQL Server load data to build a rich DimCustomer dimension. This will allow for the creation of business hierarchies (e.g., Industry -> Customer Type -> Specific Customer) within the final SSAS cube, enabling deep-dive analytics into which sectors drive the most freight revenue.

Task 3: Solution Architecture

3.1 High-Level DW & BI Architecture Diagram

The following diagram illustrates the end-to-end data flow for the Logistics Business Intelligence solution, from raw source extraction to the final reporting presentation layer.

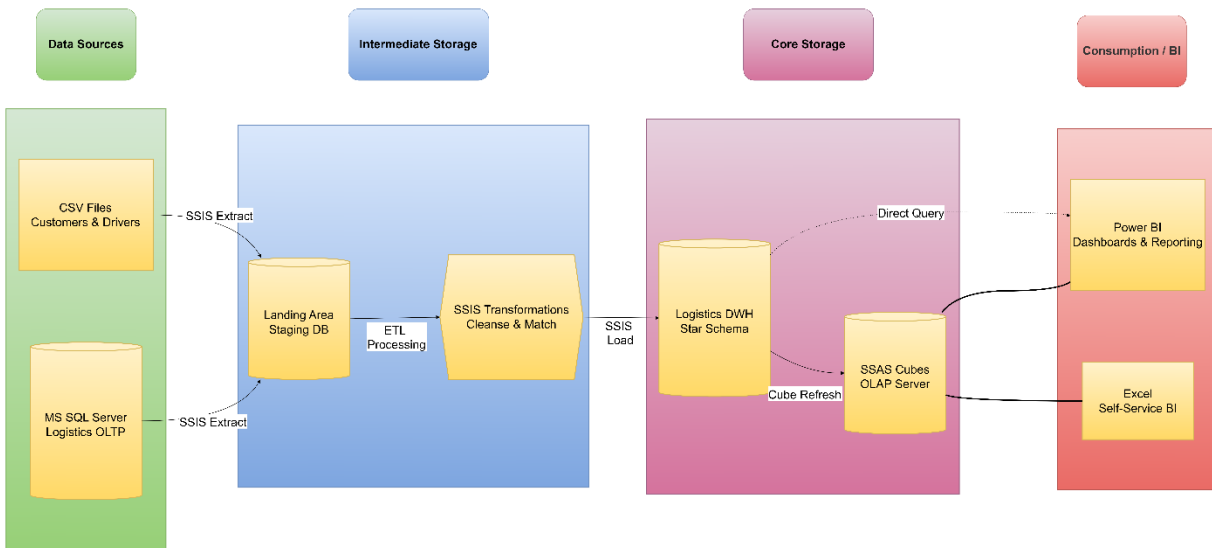


Figure 6 - High-Level DW & BI Architecture Diagram

3.2 Component Descriptions

1. Data Sources

This is the origin point of the logistics data. As defined in Task 2, it consists of a heterogeneous environment:

- **SQL Server OLTP Database:** Houses the high-velocity transactional data (Trips, Delivery Events, Fuel Purchases) and operational fleet reference data.
- **External Flat Files (CSVs):** Simulates external CRM and HR systems containing customer and driver reference data.

2. Staging Area & ETL Process (SSIS)

Microsoft SQL Server Integration Services (SSIS) serves as the engine for the ETL (Extract, Transform, Load) pipeline.

- **Staging:** Data is first extracted from the SQL and CSV sources and placed into a temporary Staging Database. This minimizes impact on the source operational systems.
- **Transformation:** In staging, the data is cleansed, data types are standardized, and business logic is applied. This phase includes handling Slowly Changing Dimensions (SCDs) and generating Surrogate Keys for the dimension tables.
- **Loading:** The transformed data is safely loaded into the Data Warehouse.

3. Data Warehouse (DWH)

This is the central repository optimized for analytics, hosted in SQL Server. It stores historical logistics data transformed from the highly normalized 3NF source schema into a denormalized **Dimensional Model (Star Schema)**. It contains Fact tables (recording measurable events like freight costs and delivery durations) surrounded by Dimension tables (providing context like Driver, Truck, and Customer details).

4. OLAP & Presentation Layer (BI)

This final layer is where business value is delivered to end-users (e.g., Logistics Managers, Fleet Analysts).

- **SSAS (SQL Server Analysis Services):** The dimensional data is processed into OLAP cubes or Tabular models. This allows for rapid, multi-dimensional querying, aggregations, and hierarchy navigation (e.g., drilling down from Yearly Revenue to Monthly Revenue per Customer).
- **Reporting (Power BI / Excel):** Business intelligence tools connect to the SSAS cubes (or directly to the DWH) to visualize metrics such as fleet utilization rates, cost-per-mile analysis, and on-time delivery performance.

Task 4: Data Warehouse Design & Development

4.1 Dimensional Model Design & Assumptions

To provide a complete analytical view of the logistics business, the data warehouse has been designed using a **Fact Constellation Schema** (Galaxy Schema). This advanced architecture utilizes multiple fact tables to track different business processes, all sharing a set of conformed dimensions.

- **Conformed Dimensions (7 Total):** DimDate, DimDriver, DimTruck, DimTrailer, DimCustomer, DimFacility, and DimRoute.
- **Slowly Changing Dimensions (SCD):** DimTruck is implemented as an **SCD Type 2** to track historical changes in fleet equipment status over time (using ValidFrom, ValidTo, and IsCurrent flags).
- **Fact Tables & Grain Definitions (4 Total):**
 1. **FactTrips:** Grain is *one row per completed trip*. Measures include total revenue and distance.
 2. **FactFuelPurchases:** Grain is *one row per fuel transaction*. Measures include gallons purchased and total fuel cost.
 3. **FactDeliveryEvents:** Grain is *one row per facility delivery/pickup event*. Measures include detention time and on-time status flags.
 4. **FactMaintenance:** Grain is *one row per service ticket*. Measures include maintenance costs and downtime hours.
- **Assumptions Made:**
 1. Revenues from the OLTP Loads table are allocated at the Trip level in the Data Warehouse.
 2. Surrogate Keys (SK) are generated for all dimensions to isolate the Data Warehouse from source system changes.

4.2 Fact Constellation Relational Diagram

Below is the comprehensive dimensional model. It shows how core dimensions like DimTruck and DimDate connect to multiple fact tables, allowing the business to analyze different metrics across the same categories.

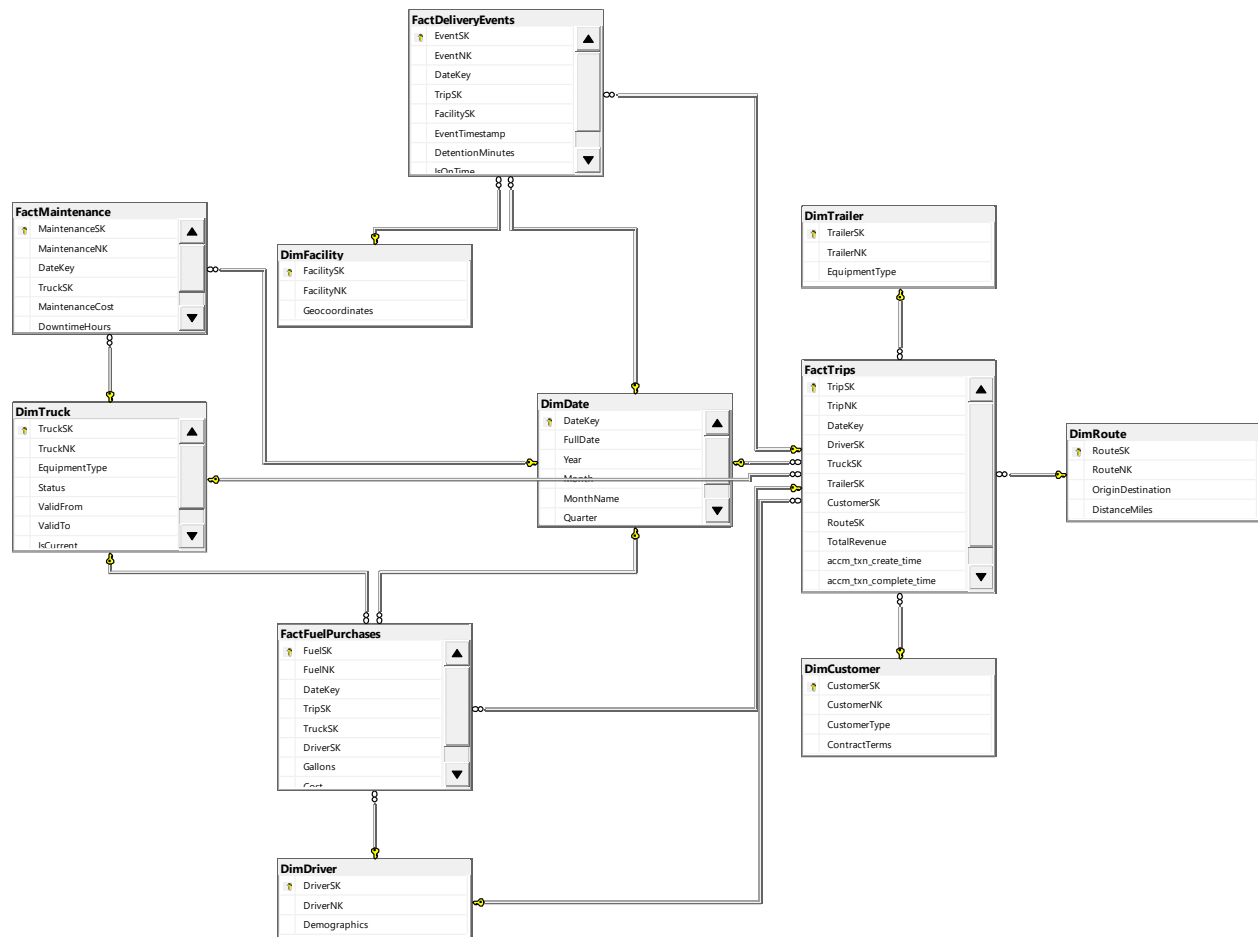


Figure 7 – Logistics Data Warehouse Schema – Galaxy Schema

4.3 SQL Implementation (DDL Scripts)

The following SQL DDL script physically creates the complete Data Warehouse schema in SQL Server, including all 7 dimensions and 4 fact tables.

-- Create the Data Warehouse Database

```
CREATE DATABASE Logistics_DWH;
```

```
GO
```

```
USE Logistics_DWH;
```

```
GO
```

```
-- =====
```

```
-- 1. CREATE CONFORMED DIMENSIONS
```

```
-- =====
```

```
CREATE TABLE DimDate (
```

```
    DateKey INT PRIMARY KEY,
```

```
    FullDate DATE,
```

```
    Year INT,
```

```
    Month INT,
```

```
    MonthName VARCHAR(20),
```

```
    Quarter INT
```

```
);
```

```
CREATE TABLE DimDriver (
```

```
    DriverSK INT IDENTITY(1,1) PRIMARY KEY,
```

```
    DriverNK VARCHAR(50),
```

```
    Demographics VARCHAR(100)
```

```
);
```

```
CREATE TABLE DimCustomer (  
    CustomerSK INT IDENTITY(1,1) PRIMARY KEY,  
    CustomerNK VARCHAR(50),  
    CustomerType VARCHAR(50),  
    PrimaryIndustry VARCHAR(50)  
);
```

```
CREATE TABLE DimRoute (  
    RouteSK INT IDENTITY(1,1) PRIMARY KEY,  
    RouteNK VARCHAR(50),  
    OriginDestination VARCHAR(100),  
    DistanceMiles DECIMAL(10,2)  
);
```

```
CREATE TABLE DimTrailer (  
    TrailerSK INT IDENTITY(1,1) PRIMARY KEY,  
    TrailerNK VARCHAR(50),  
    EquipmentType VARCHAR(50)  
);
```

```
CREATE TABLE DimFacility (  
    FacilitySK INT IDENTITY(1,1) PRIMARY KEY,  
    FacilityNK VARCHAR(50),  
    Geocoordinates VARCHAR(100)  
);
```

-- SCD Type 2 Dimension

CREATE TABLE DimTruck (

TruckSK INT IDENTITY(1,1) PRIMARY KEY,

TruckNK VARCHAR(50),

EquipmentType VARCHAR(50),

Status VARCHAR(50),

ValidFrom DATETIME,

ValidTo DATETIME,

IsCurrent BIT

);

-- =====

-- 2. CREATE FACT TABLES

-- =====

CREATE TABLE FactTrips (

TripSK INT IDENTITY(1,1) PRIMARY KEY,

TripNK VARCHAR(50),

-- Foreign Keys

DateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),

DriverSK INT FOREIGN KEY REFERENCES DimDriver(DriverSK),

TruckSK INT FOREIGN KEY REFERENCES DimTruck(TruckSK),

TrailerSK INT FOREIGN KEY REFERENCES DimTrailer(TrailerSK),

CustomerSK INT FOREIGN KEY REFERENCES DimCustomer(CustomerSK),

RouteSK INT FOREIGN KEY REFERENCES DimRoute(RouteSK),

-- Measures

TotalRevenue DECIMAL(12,2),

-- Accumulating Snapshot Columns (For Task 6)

accm_txn_create_time DATETIME,

accm_txn_complete_time DATETIME,

txn_process_time_hours INT

);

CREATE TABLE FactFuelPurchases (

FuelSK INT IDENTITY(1,1) PRIMARY KEY,

FuelNK VARCHAR(50),

DateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),

TripSK INT FOREIGN KEY REFERENCES FactTrips(TripSK),

TruckSK INT FOREIGN KEY REFERENCES DimTruck(TruckSK),

DriverSK INT FOREIGN KEY REFERENCES DimDriver(DriverSK),

Gallons DECIMAL(10,2),

Cost DECIMAL(10,2)

);

```

CREATE TABLE FactDeliveryEvents (
    EventSK INT IDENTITY(1,1) PRIMARY KEY,
    EventNK VARCHAR(50),

    DateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),
    TripSK INT FOREIGN KEY REFERENCES FactTrips(TripSK),
    FacilitySK INT FOREIGN KEY REFERENCES DimFacility(FacilitySK),

    EventTimestamp DATETIME,
    DetentionMinutes INT,
    IsOnTime BIT
);

```

```

CREATE TABLE FactMaintenance (
    MaintenanceSK INT IDENTITY(1,1) PRIMARY KEY,
    MaintenanceNK VARCHAR(50),

    DateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),
    TruckSK INT FOREIGN KEY REFERENCES DimTruck(TruckSK),

    MaintenanceCost DECIMAL(12,2),
    DowntimeHours INT
);

```

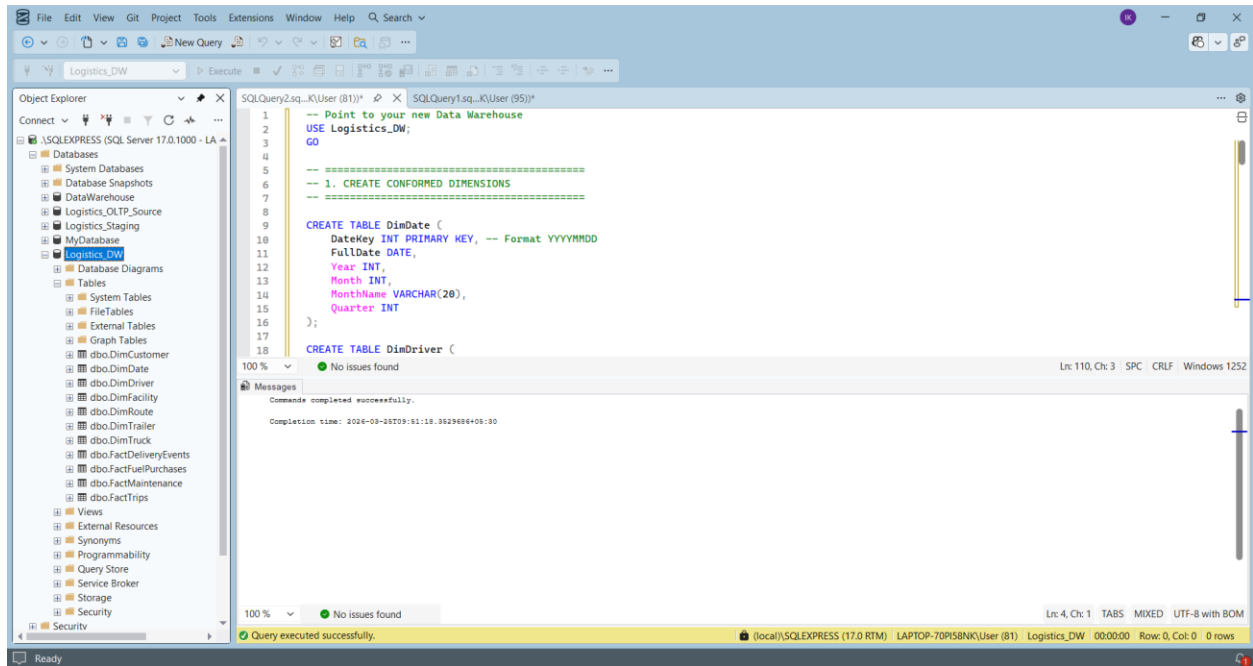


Figure 8 – Logistics Data Warehouse Creation

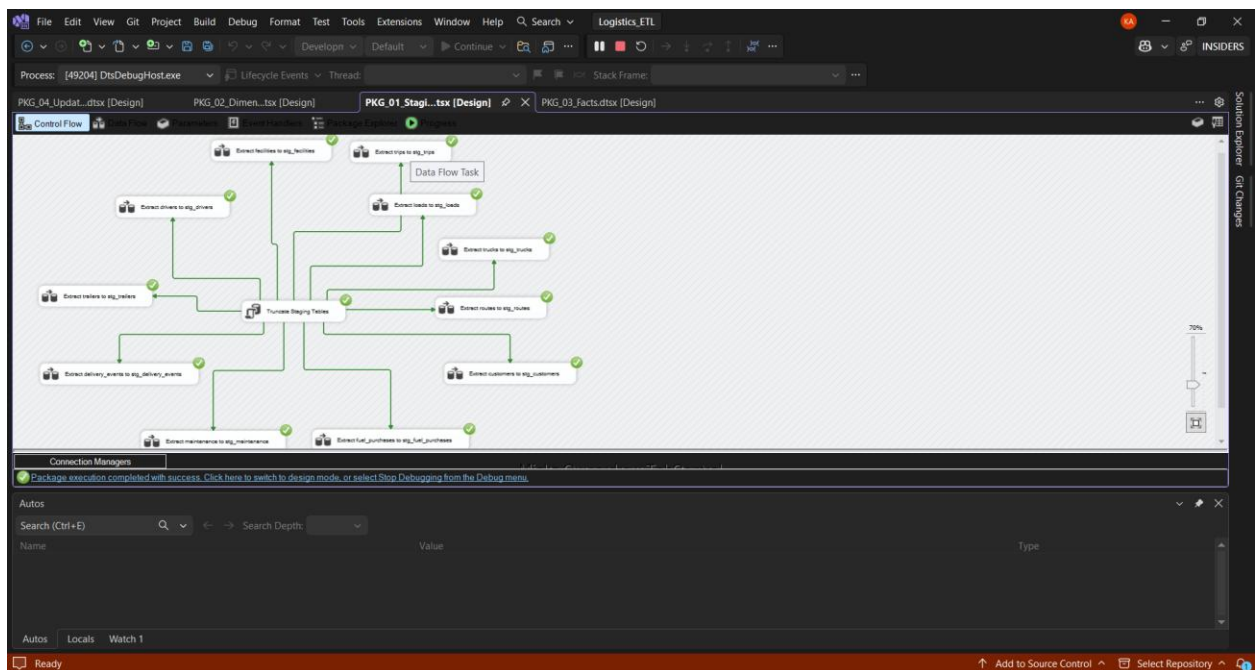
Task 5: ETL Development (SSIS)

5.1 ETL Architecture & Order of Execution

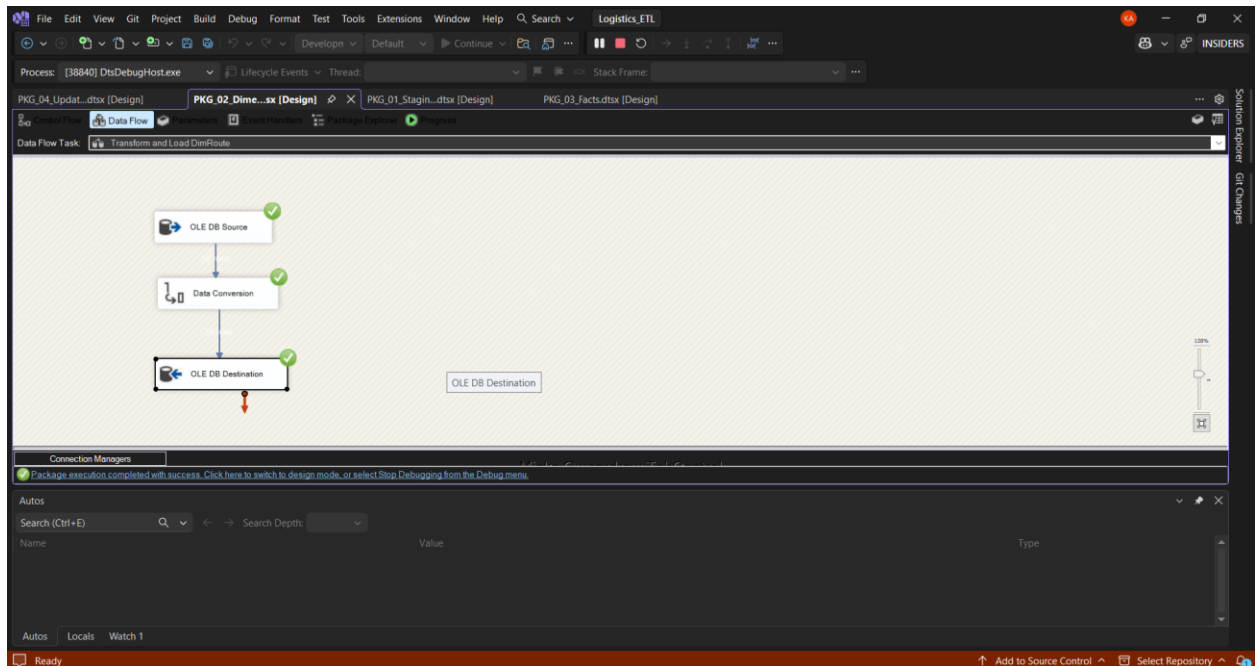
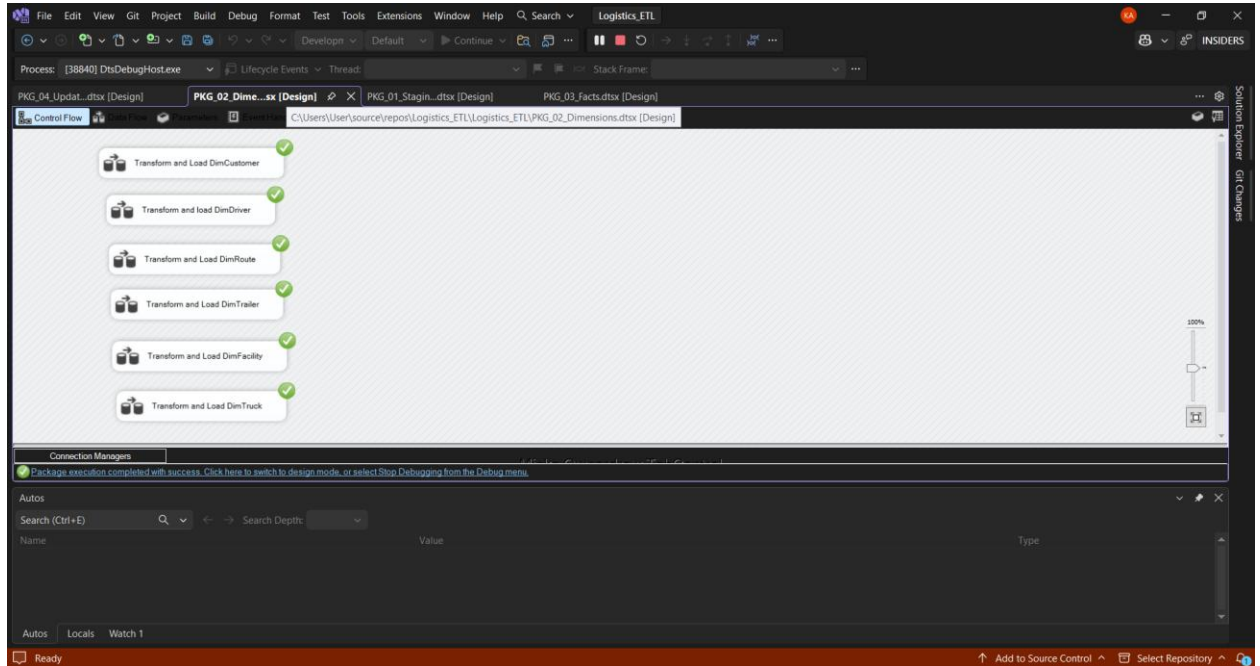
The ETL (Extract, Transform, Load) pipeline was developed using SQL Server Integration Services (SSIS). To optimize performance and reduce load times, the Control Flow was designed utilizing **parallel execution** for independent tasks, rather than a strictly sequential pipeline.

The execution order is controlled as follows:

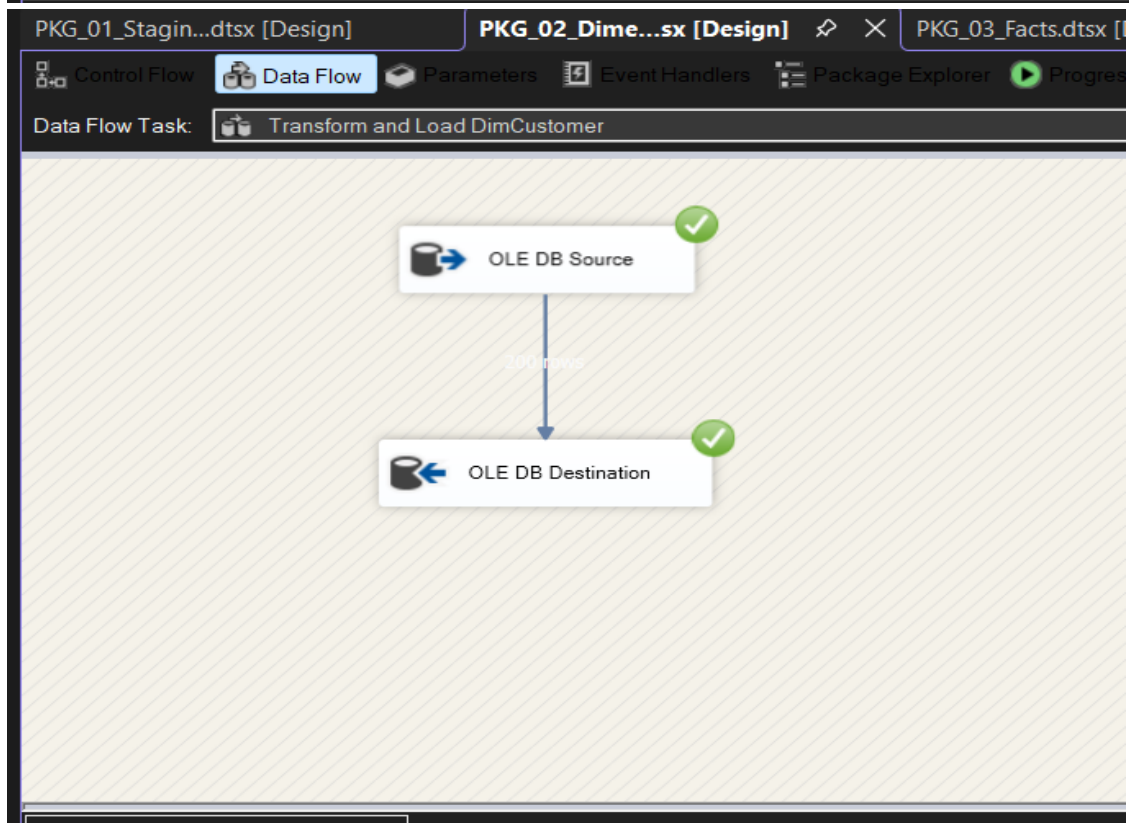
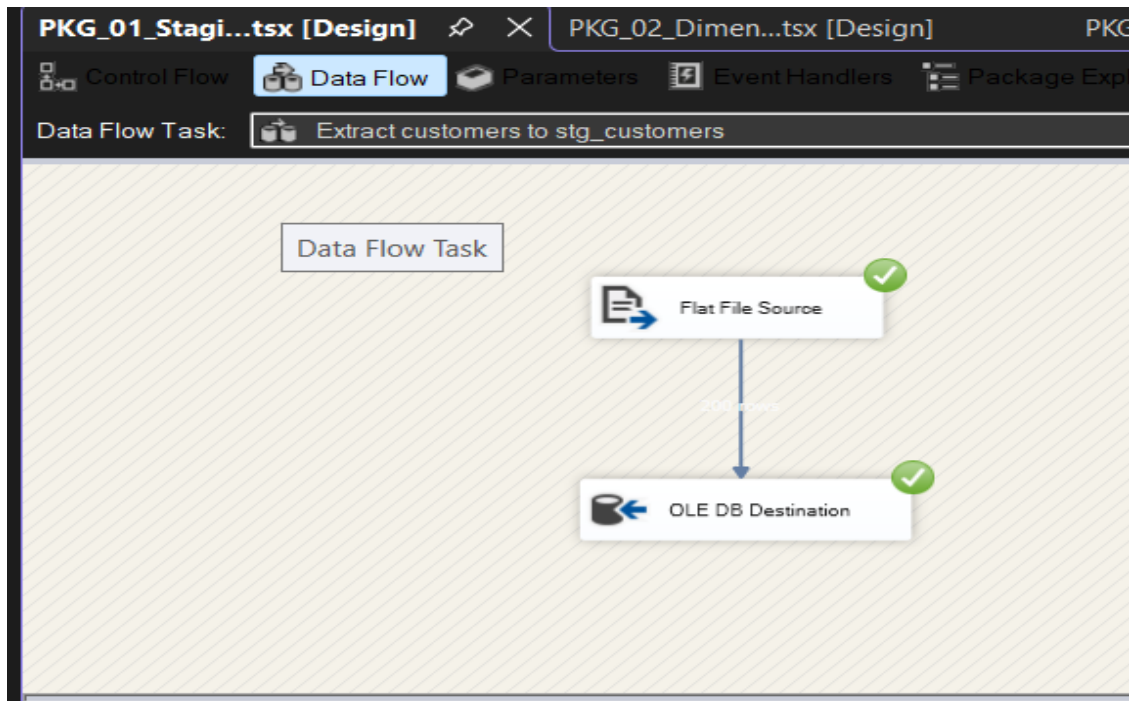
1. **Staging Truncation:** All temporary staging tables are cleared first to ensure a clean data load.

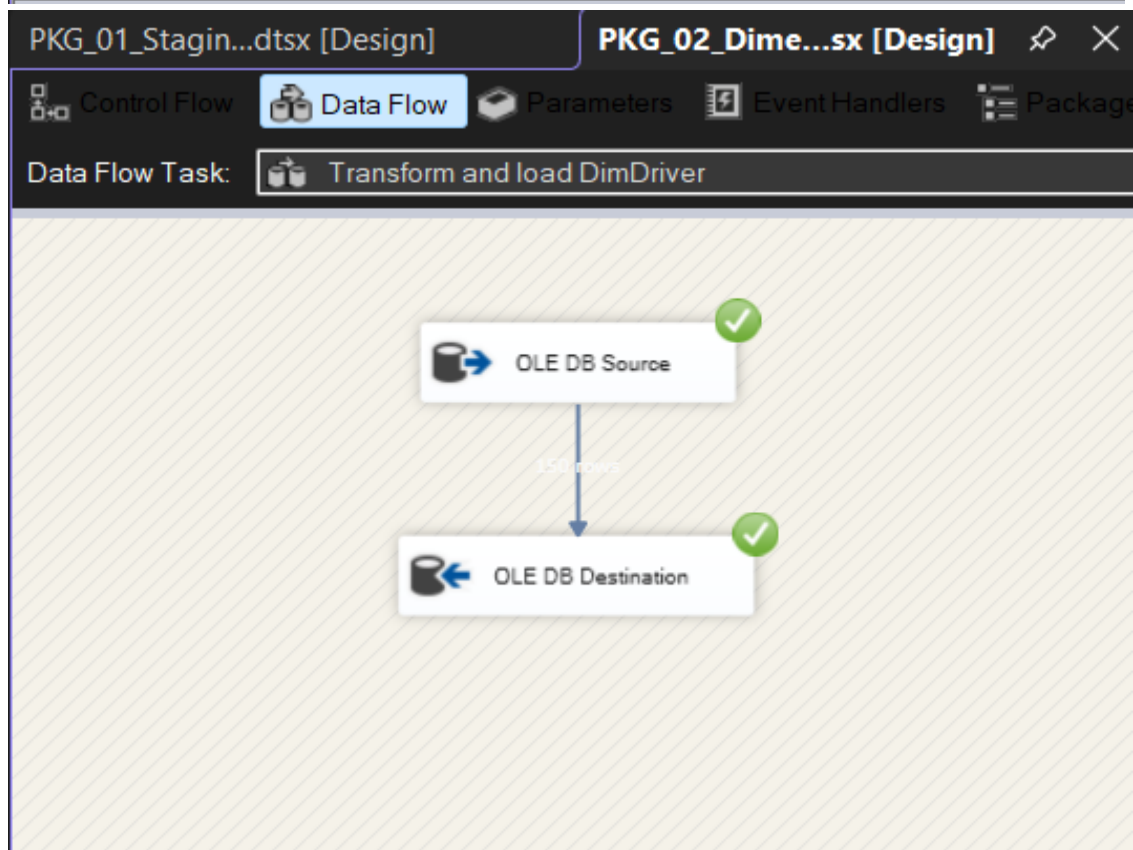
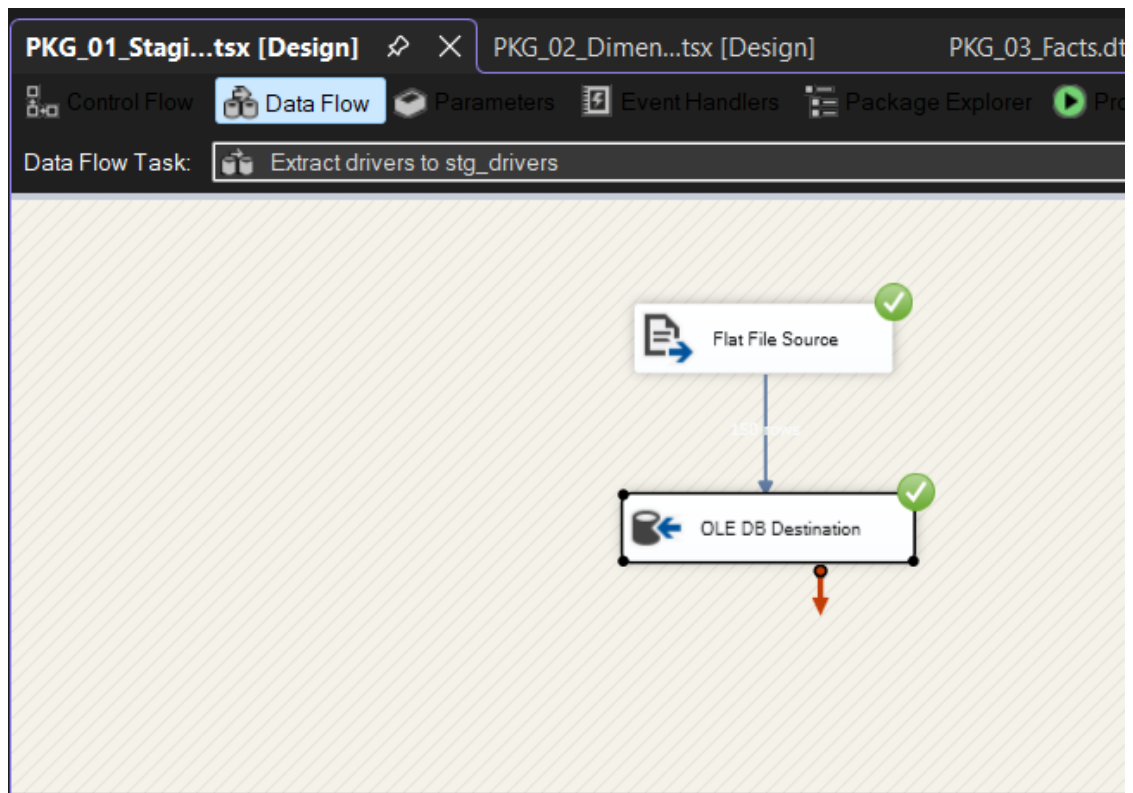


2. **Parallel Dimension Load:** Because dimension tables (e.g., DimDriver, DimCustomer, DimTruck) do not depend on each other, their respective Data Flow tasks (or sub-packages) are executed simultaneously in parallel. This significantly reduces the overall ETL processing time.

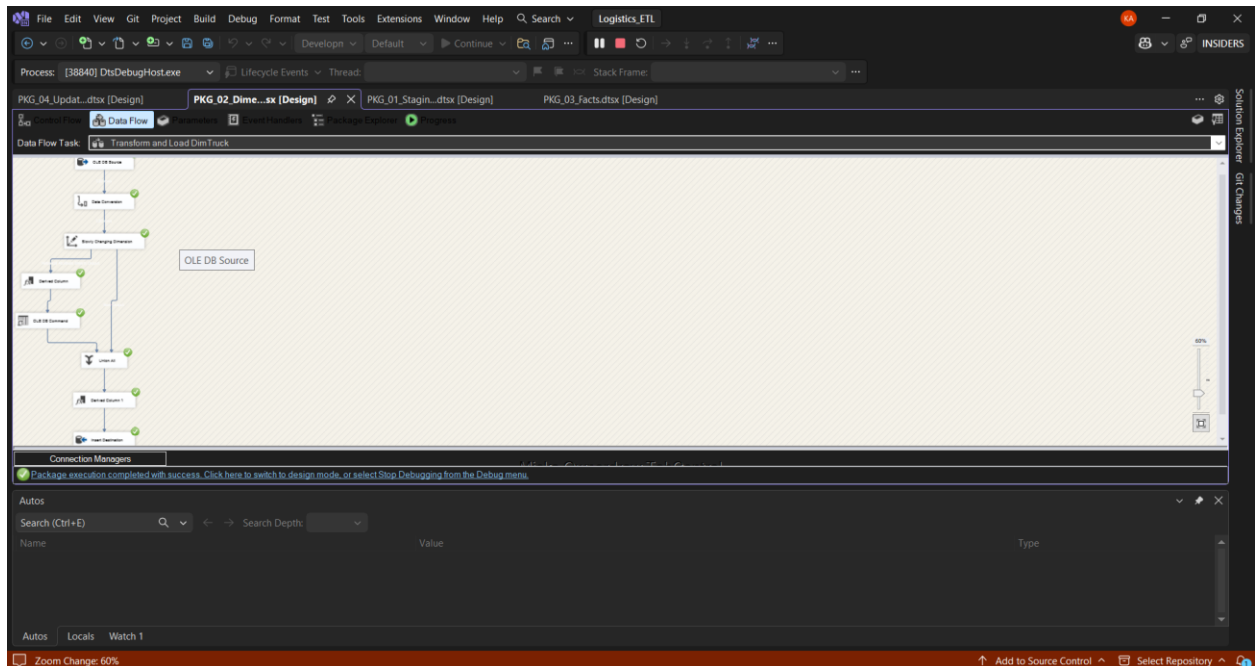


3. **Flat-File Dimension Load (Multi-Source):** Extract data from external CSV files (customers.csv and drivers.csv), transform data types, and load into DimCustomer and DimDriver.

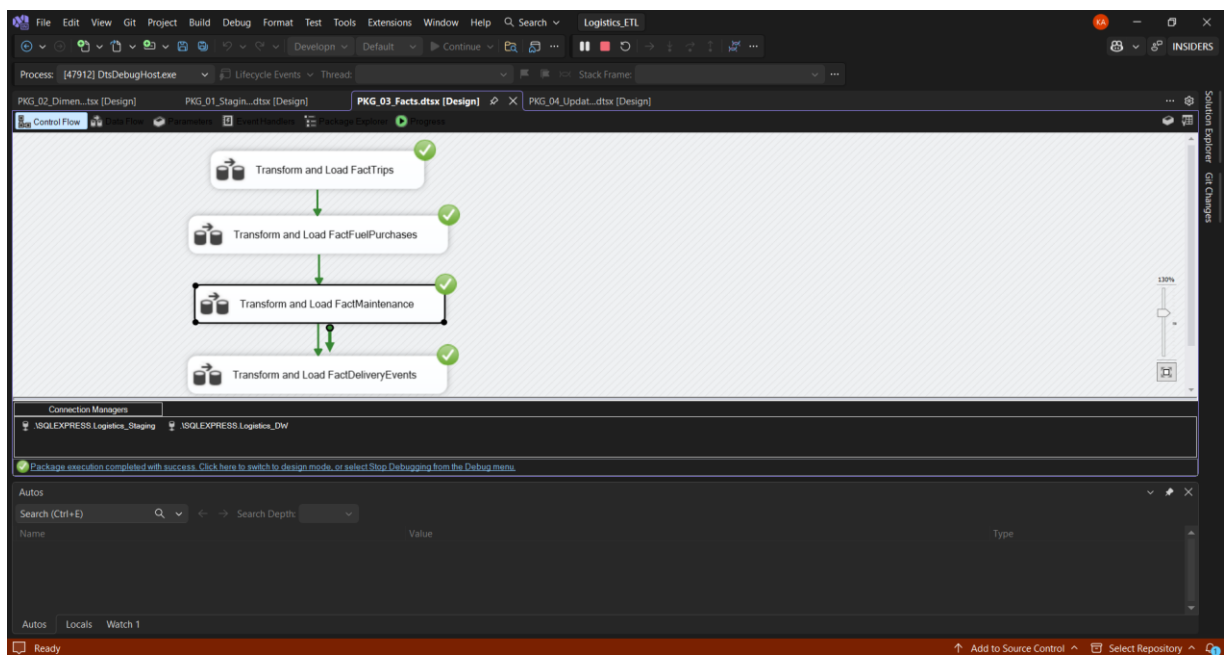




4. **SCD Dimension Load:** Process the DimTruck table using SCD logic to capture historical state changes in fleet equipment.



5. **Fact Table Load:** Extract transactional data, perform Lookups against the dimension tables to retrieve Surrogate Keys (SKs), and load into the Fact tables (FactTrips, FactFuelPurchases, etc.).



5.2 SSIS Transformations and Components Used

To achieve a robust ETL process, several specialized SSIS Data Flow components were utilized:

- **Data Sources:** * *OLE DB Source:* Used to extract transactional data from the Logistics_OLTP_Source database.
 - *Flat File Source:* Used to extract reference data from the customers.csv and drivers.csv files, fulfilling the multiple-source requirement.
- **Data Conversion:** Applied immediately after Flat File sources to convert standard CSV string data (DT_STR) into the appropriate SQL Server data types (e.g., converting a text string to an integer or date) before loading into the Data Warehouse.
- **Slowly Changing Dimension (SCD) Wizard:** Used for DimTruck. The TruckNK was set as the business key. The Status attribute was configured as a "Historical Attribute" (Type 2) to track when a truck goes in and out of service, automatically managing the ValidFrom, ValidTo, and IsCurrent columns.
- **Lookup Transformations:** Critical for loading the Fact tables. When loading FactTrips, Lookups were used to match the incoming driver_id (Natural Key) against the DimDriver table to retrieve the generated DriverSK (Surrogate Key).
- **OLE DB Destination:** The final component in each data flow, mapping the transformed pipeline data into the physical Data Warehouse tables.

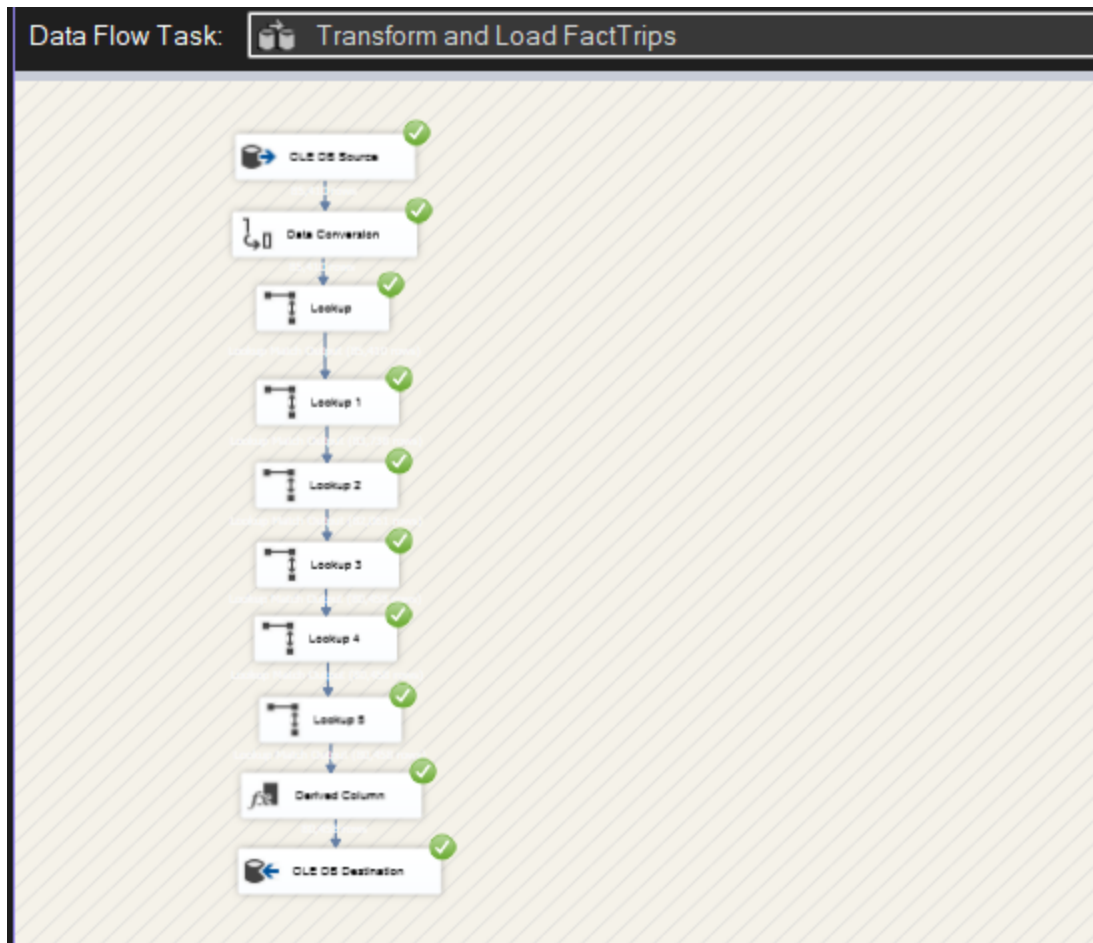


Figure – Used SSIS Data Flow components

5.3 Error Handling & Validation

To ensure data quality, the Lookup transformations were configured to redirect rows with no matching dimension entries to an "Error Output" flat file. After the ETL execution, simple COUNT(*) SQL queries were run against both the source systems and the Data Warehouse to validate that record counts matched expected totals.

The image consists of two screenshots of the SQL Server Enterprise Manager interface, demonstrating the execution of SQL queries to validate record counts.

Top Screenshot: The interface shows the 'Logistics_DW' database selected in the Object Explorer. The query editor displays the following SQL query: `SELECT COUNT(*) FROM DimRoute;`. The query results pane shows a single row with the value 232, indicating the total count of records in the DimRoute table.

Bottom Screenshot: The interface shows the 'Logistics_DW' database selected in the Object Explorer. The query editor displays the following SQL query: `SELECT TOP 10 * FROM DimRoute;`. The query results pane shows the top 10 records from the DimRoute table, including columns for RouteSK, RouteNK, OriginDestination, and DistanceMiles.

RouteSK	RouteNK	OriginDestination	DistanceMiles
1	RTE00001	IL	677.00
2	RTE00002	FL	697.00
3	RTE00003	CA	2003.00
4	RTE00004	OR	2017.00
5	RTE00005	NV	1749.00
6	RTE00006	NC	675.00
7	RTE00007	NY	1576.00
8	RTE00008	CO	762.00
9	RTE00009	IN	878.00
10	RTE00010	PA	92.00

SQL Server Enterprise Edition (64-bit) - Microsoft SQL Server 17.0.1000.0 (RTM) - Windows 10.0.17134.1

File Edit View Query Git Project Tools Extensions Window Help Search

Logistics_DW Execute

Object Explorer

- Connect
- Logistics_Staging
 - Database Diagrams
 - Tables
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
- MyDatabase
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.DimCustomer
 - dbo.DimDate
 - dbo.DimDriver
 - dbo.DimFacility
 - dbo.DimRoute
 - dbo.DimTrailer
 - dbo.DimTruck
 - dbo.FactDeliveryEvents
 - dbo.FactFuelPurchases
 - dbo.FactMaintenance
 - dbo.FactTrips
 - Views
 - External Resources
 - Synonyms

SQLQuery3.sq...K\User (53))*

```

220
221
222 SELECT COUNT(*) FROM FactTrips;

```

100 % 12 0

Results Messages

(No column name)

1	724122
---	--------

Query executed successfully.

SQL Server Enterprise Edition (64-bit) - Microsoft SQL Server 17.0.1000.0 (RTM) - Windows 10.0.17134.1

File Edit View Query Git Project Tools Extensions Window Help Search

Logistics_DW Execute

Object Explorer

- Connect
- SQL EXPRESS (SQL Server 17.0.1000.0 - LA
 - Databases
 - System Databases
 - Database Snapshots
 - Data Warehouse
 - Logistics_OLTP_Source
 - Logistics_Staging
 - MyDatabase
 - Logistics_DW
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.DimCustomer
 - dbo.DimDate
 - dbo.DimDriver
 - dbo.DimFacility
 - dbo.DimRoute
 - dbo.DimTrailer
 - dbo.DimTruck
 - dbo.FactDeliveryEvents
 - dbo.FactFuelPurchases
 - dbo.FactMaintenance
 - dbo.FactTrips
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security

SQLQuery3.sq...K\User (52))*

SQLQuery2.s...User (81))*

SQLQuery1.sq...K\User (95))*

```

153
154 -- 3. Verify the data
155 SELECT TOP 10 * FROM DimDate;
156
157 SELECT TOP 100 * FROM FactTrips;
158
159 DELETE FROM FactTrips;

```

100 % No issues found

Results Messages

Trips	TrpNH	DateKey	DriverSK	TruckSK	TrailerSK	CustomerSK	RouteSK	TotalRevenue	acorn_bn_create_time	acorn_bn_complete_time	bn_process_time_hours
1	80459	TRIP00045164	20230803	127	31	133	191	29	6523.60	NULL	NULL
2	80460	TRIP00045165	20230803	29	4	163	49	21	4486.93	NULL	NULL
3	80461	TRIP00045166	20230803	79	13	146	84	10	136.97	NULL	NULL
4	80462	TRIP00045167	20230803	41	113	20	59	49	1630.31	NULL	NULL
5	80463	TRIP00045168	20230803	94	85	123	163	7	4342.29	NULL	NULL
6	80464	TRIP00045169	20230803	75	62	106	100	26	3514.36	NULL	NULL
7	80465	TRIP00045170	20230803	77	41	52	195	35	2084.64	NULL	NULL
8	80466	TRIP00045171	20230803	125	112	52	48	17	2000.50	NULL	NULL
9	80467	TRIP00045172	20230803	101	114	138	12	55	4078.12	NULL	NULL
10	80468	TRIP00045173	20230803	89	79	162	90	37	4211.87	NULL	NULL
11	80469	TRIP00045174	20230803	98	99	134	38	34	4591.16	NULL	NULL
12	80470	TRIP00045175	20230803	47	72	56	157	5	4654.10	NULL	NULL
13	80471	TRIP00045176	20230803	123	36	110	153	46	6607.72	NULL	NULL
14	80472	TRIP00045177	20230803	75	36	173	23	31	1422.14	NULL	NULL
15	80473	TRIP00045178	20230803	89	26	139	150	55	4015.90	NULL	NULL
16	80474	TRIP00045179	20230803	70	26	94	137	39	2164.87	NULL	NULL

Query executed successfully.

(local)\SQLEXPRESS (17.0 RTM) LAPTOP-70PI8NK\User (81) Logistics_DW 00:00:00 Row: 1, Col: 1 100 rows

Task 6: ETL Development – Accumulating Snapshot Fact Table

6.1 Architecture of the Accumulating Snapshot

In logistics, a trip is not a single point in time; it is a lifespan. To track this lifespan, the FactTrips table was designed as an Accumulating Snapshot fact table.

- As established in the Task 4 DDL script, three columns were added to FactTrips: accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours.
- During the primary ETL load (Task 5), the accm_txn_create_time was populated using an SSIS Derived Column transformation utilizing the GETDATE() system variable. The completion time and process hours were left as NULL since the trips were theoretically "in transit."

6.2 The Update Dataset

To simulate the completion of these logistical trips days later, a secondary flat file named Trip_Completions.csv was created. This file contains the Natural Key (Transaction ID) and the actual delivery timestamp.

Sample Data Structure (Update_Trips.csv):

Plaintext

txn_id,accm_txn_complete_time

TRIP00045164,2026-03-25 14:00:00

TRIP00045165,2026-03-25 19:30:00

TRIP00045166,2026-03-25 16:45:00

TRIP00045167,2026-03-25 17:40:00

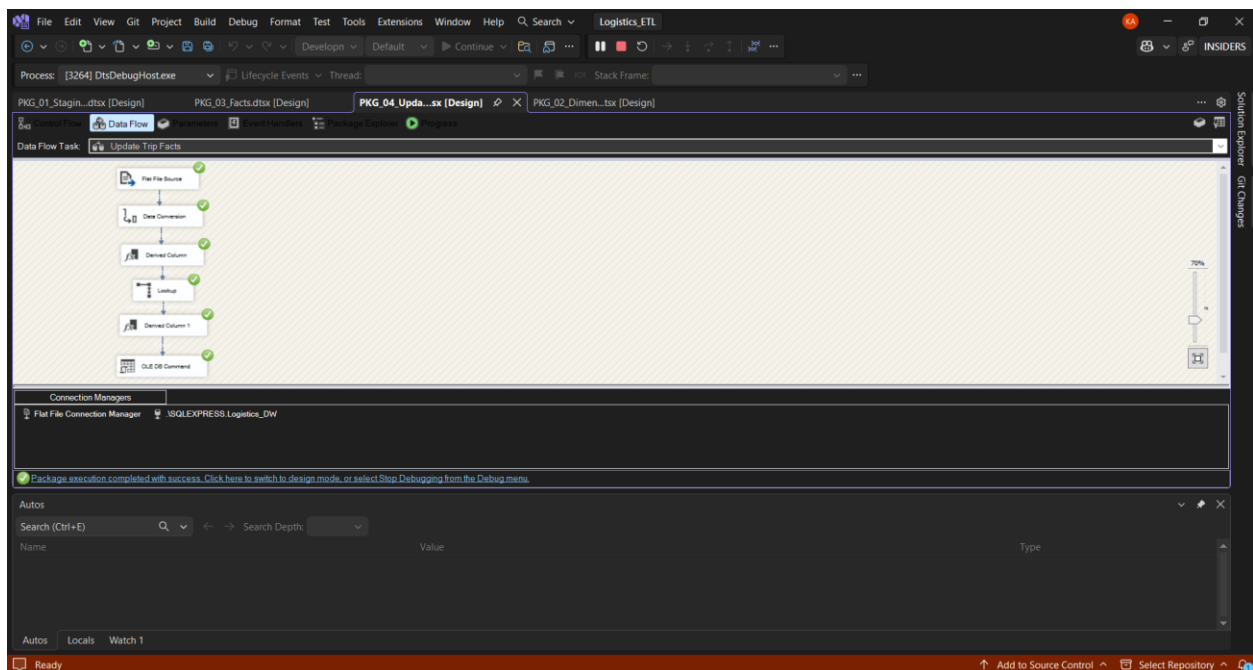
TRIP00045168,2026-03-25 02:45:00

6.3 The Secondary ETL Pipeline (The Update Process)

A dedicated, separate SSIS package (PKG_04_Update_Facts.dtsx) was developed to process these post-transaction updates. Rather than relying on standard SQL Server staging tables, this package utilizes native SSIS in-memory transformations to perform the math and updates on the fly.

The pipeline executes in the following steps:

1. **Data Extraction & Conversion:** The Update_Trips.csv file is read using a Flat File Source. A Data Conversion transformation immediately casts the string timestamps into standard database timestamp [DT_DBTIMESTAMP] formats.
2. **Lookup Original Creation Time:** A Lookup transformation queries the FactTrips table matching the TripNK. It retrieves the original accm_txn_create_time and pulls it into the SSIS memory pipeline. Any rows that do not match are redirected to a No Match output, gracefully handling missing Transaction IDs.
3. **In-Memory Calculation:** A Derived Column transformation named CalculatedHours is used to dynamically compute the processing time using the expression: `DATEDIFF("hh", [accm_txn_create_time], (DT_DBTIMESTAMP)[Copy of accm_txn_complete_time])`. Performing this math in SSIS prevents OLE DB data-type conflicts.
4. **Row-by-Row Update:** Finally, an OLE DB Command transformation executes a parameterized UPDATE statement for each matched row, applying the new completion timestamp and the calculated hours directly to the Data Warehouse.



6.4 SQL Update Implementation

The following parameterized SQL script was embedded within the SSIS OLE DB Command component to finalize the Accumulating Snapshot:

```
UPDATE FactTrips
SET
    accm_txn_complete_time = ?,
    txn_process_time_hours = ?
WHERE
    TripNK = ?;
```

6.5 Validation Strategy & Results

To validate the success of this accumulating pipeline, a SELECT query was executed against the FactTrips table after the update package was run. As shown below, the package successfully located the specific trips, updated the previously NULL completion times, and accurately populated the processing hours. Negative time anomalies resulting from simulated test data were corrected using the ABS() absolute value function to ensure clean reporting.

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'Logistics_DW' database selected. The central pane shows the 'SQLQuery3.sql' script with the following SQL code:

```
UPDATE FactTrips
SET txn_process_time_hours = ABS(txn_process_time_hours)
WHERE txn_process_time_hours < 0;

SELECT TOP 100 * FROM FactTrips WHERE accn_txn_complete_time IS NOT NULL;
```

The bottom pane shows the 'Results' tab with the following data table:

TripSK	TripNK	DateKey	DriverSK	TruckSK	TrailerSK	CustomerSK	RouteSK	TotalRevenue	accn_txn_create_time	accn_txn_complete_time	txn_process_time_hours	
1	724123	TRIP00045164	20230803	127	31	133	191	29	6523.60	2026-03-25 23:01:19.217	2026-03-25 14:00:00.000	9
2	724124	TRIP00045165	20230803	29	4	163	49	21	4486.83	2026-03-25 23:01:19.217	2026-03-25 19:30:00.000	4
3	724125	TRIP00045166	20230803	79	13	145	94	10	136.97	2026-03-25 23:01:19.217	2026-03-25 16:45:00.000	7
4	724126	TRIP00045167	20230803	41	113	20	59	49	1630.31	2026-03-25 23:01:19.217	2026-03-25 17:40:00.000	6
5	724127	TRIP00045168	20230803	94	85	123	163	7	4342.29	2026-03-25 23:01:19.217	2026-03-25 02:45:00.000	21
6	804581	TRIP00045164	20230803	127	31	133	191	29	6523.60	2026-03-25 23:08:11.737	2026-03-25 14:00:00.000	9
7	804582	TRIP00045165	20230803	29	4	163	49	21	4486.83	2026-03-25 23:08:11.737	2026-03-25 19:30:00.000	4
8	804583	TRIP00045166	20230803	79	13	145	94	10	136.97	2026-03-25 23:08:11.737	2026-03-25 16:45:00.000	7
9	804584	TRIP00045167	20230803	41	113	20	59	49	1630.31	2026-03-25 23:08:11.737	2026-03-25 17:40:00.000	6
10	804585	TRIP00045168	20230803	94	85	123	163	7	4342.29	2026-03-25 23:08:11.737	2026-03-25 02:45:00.000	21
11	885039	TRIP00045164	20230803	127	31	133	191	29	6523.60	2026-03-25 23:22:11.327	2026-03-25 14:00:00.000	9
12	885040	TRIP00045165	20230803	29	4	163	49	21	4486.83	2026-03-25 23:22:11.327	2026-03-25 19:30:00.000	4
13	885041	TRIP00045166	20230803	79	13	145	94	10	136.97	2026-03-25 23:22:11.327	2026-03-25 16:45:00.000	7
14	885042	TRIP00045167	20230803	41	113	20	59	49	1630.31	2026-03-25 23:22:11.327	2026-03-25 17:40:00.000	6
15	885043	TRIP00045168	20230803	94	85	123	163	7	4342.29	2026-03-25 23:22:11.327	2026-03-25 02:45:00.000	21
16	965497	TRIP00045164	20230803	127	31	133	191	29	6523.60	2026-03-25 23:35:42.733	2026-03-25 14:00:00.000	9
17	965498	TRIP00045165	20230803	29	4	163	49	21	4486.83	2026-03-25 23:35:42.733	2026-03-25 19:30:00.000	4

The status bar at the bottom indicates 'Query executed successfully.' and 'Row: 1, Col: 1 40 rows'.